



國立清華大學
NATIONAL TSING HUA UNIVERSITY

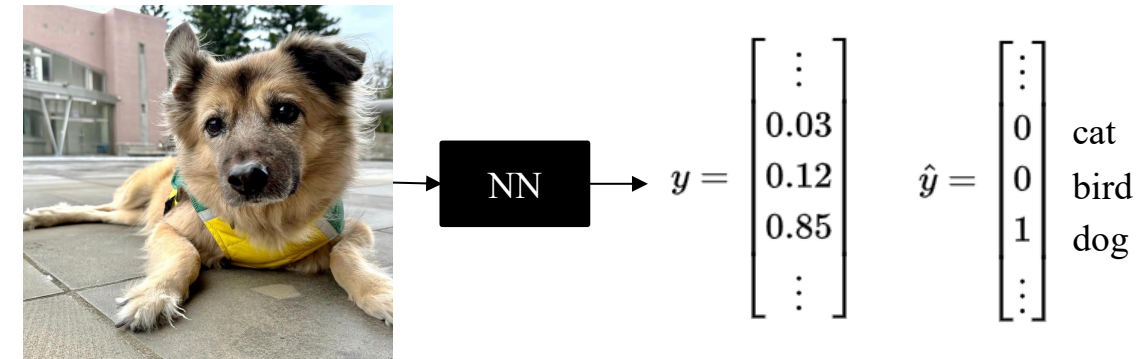
Deep Learning and Applications

Ching-I Huang



Challenges Faced by Deep Neural Networks (DNNs)

- Problem Overview
 1. Traditional DNNs use **fully connected layers**
 2. Every output unit interacts with **all input units**



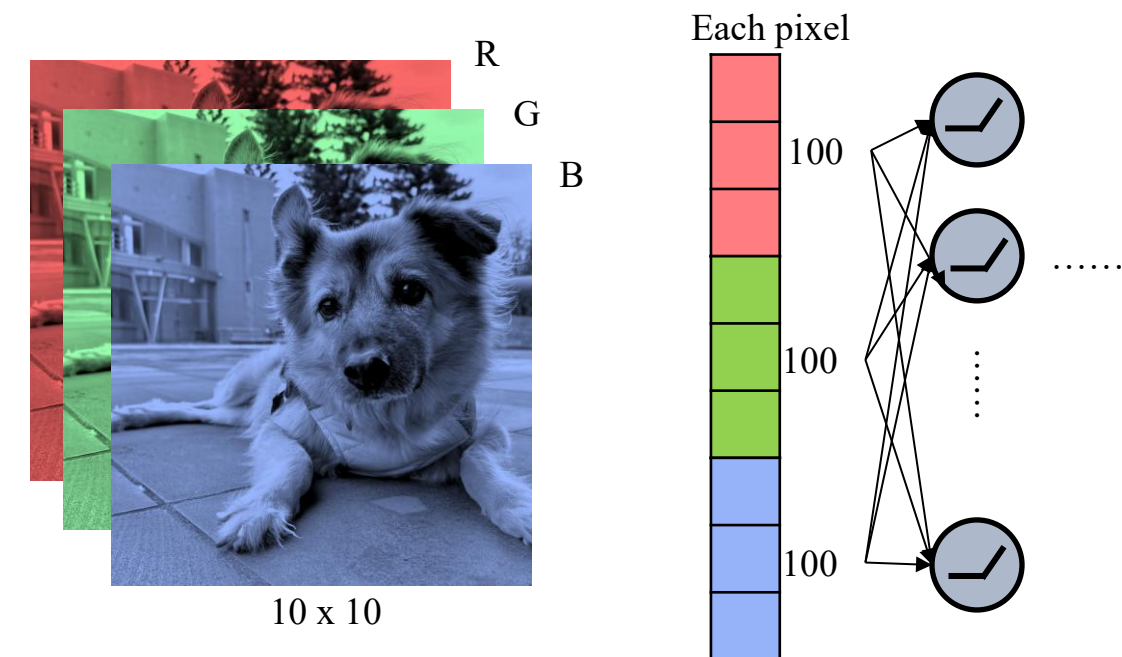
- This leads to:
 1. **Massive number of parameters**
 2. **High computational cost**
 3. Poor statistical efficiency

When the training data is limited, the model may overfit—memorizing the training examples rather than learning generalizable patterns.

4. Inflexibility with input size

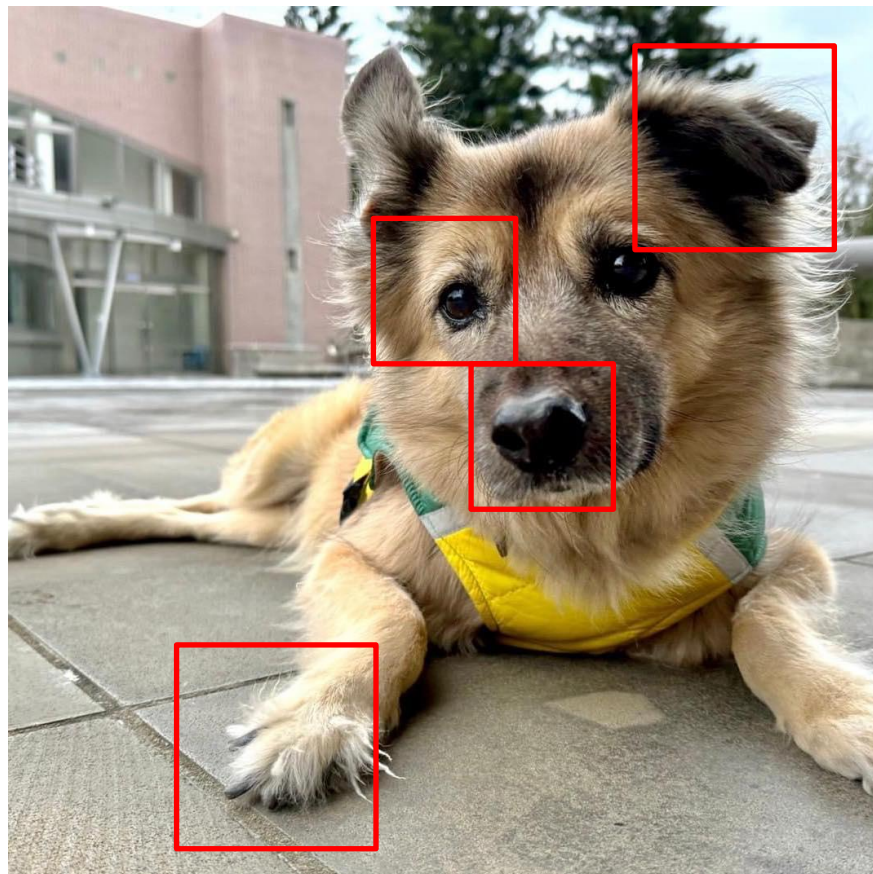
$$x \in \mathbb{R}^{100} \rightarrow W \in \mathbb{R}^{m \times 100}$$

→ The **input dimension must match the weight matrix**

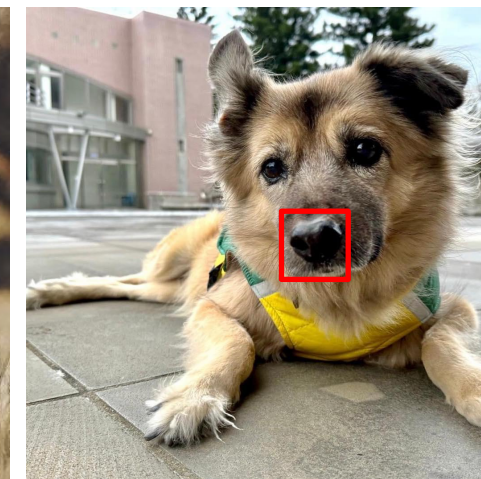




Why CNN? (Convolutional Neural Networks)



- The patterns are smaller than the entire image.
 - The same patterns appear in different regions.
- Can we tie the weights for certain groups and prune several weights?

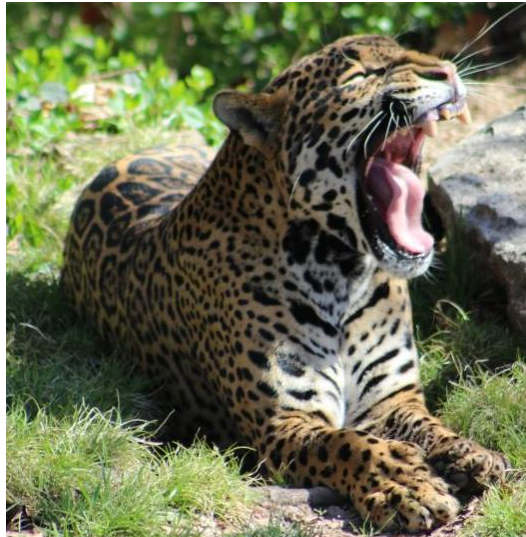


Parameter Sharing?

The set of neurons may be with the same set of parameter.



Classification



Jaguar美洲豹 (*Panthera onca*)



Leopard花豹 (*Panthera pardus*)



Cheetah獵豹 (*Acinonyx jubatus*)



Clouded Leopard雲豹 (*Neofelis nebulosa*)

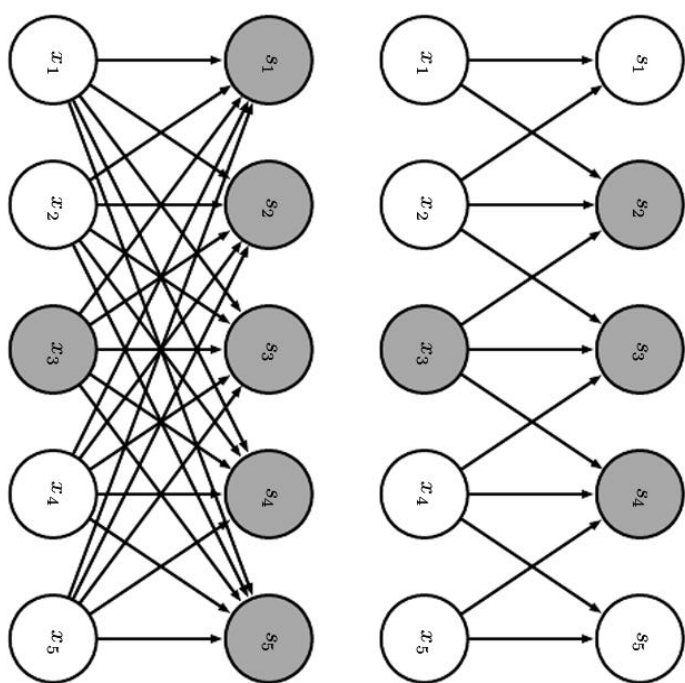
Feature	Cheetah	Leopard
Facial Markings	Has distinctive black “tear lines” running from the inner corners of the eyes down to the mouth	No tear lines on the face
Spot Pattern	Solid, small black spots	Rosette-shaped spots (rosettes) with hollow centers
Body Build	Slim and slender, with long legs	Stocky and muscular, with shorter, powerful legs
Taxonomic Classification	Family: Felidae; Subfamily: Felinae	Family: Felidae; Subfamily: Pantherinae
Running Speed	Extremely fast; can reach speeds of 110–120 km/h (68–75 mph)	Agile and strong, but not built for extreme speed
Habitat	Primarily found in grasslands of Africa and parts of Asia	Wide distribution; inhabits forests, grasslands, deserts across Asia, Africa, and the Middle East
Special Abilities	Exceptional speed for sprinting	Skilled climber, swimmer, and can dive



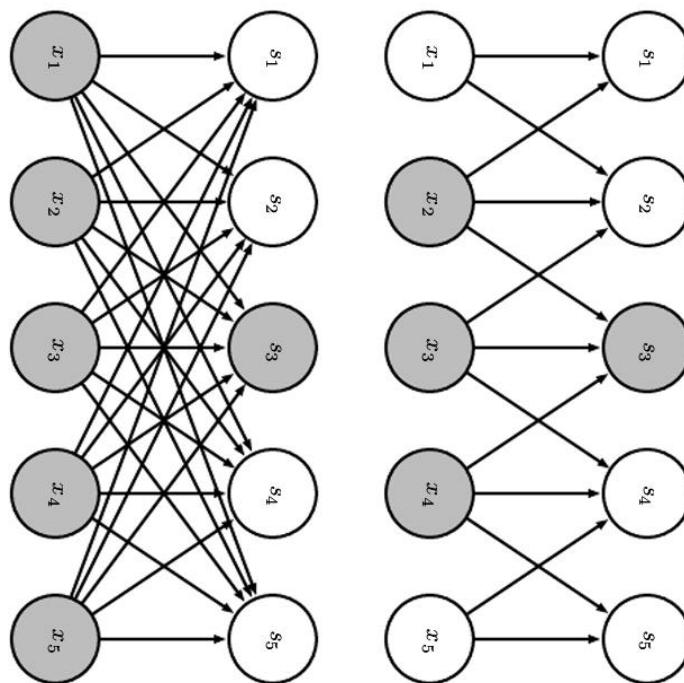
CNN's Key Ideas (9.3)

- Sparse Connectivity

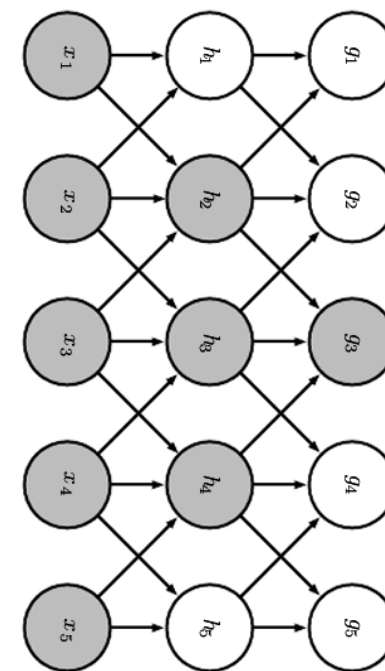
1. Each neuron connects only to a **local region** of the input, rather than all input units (as in fully connected layers).
2. Reduces the number of parameters → **less memory** and **faster computation**.
3. Focuses on meaningful **local features**



convolution with a kernel of width 3

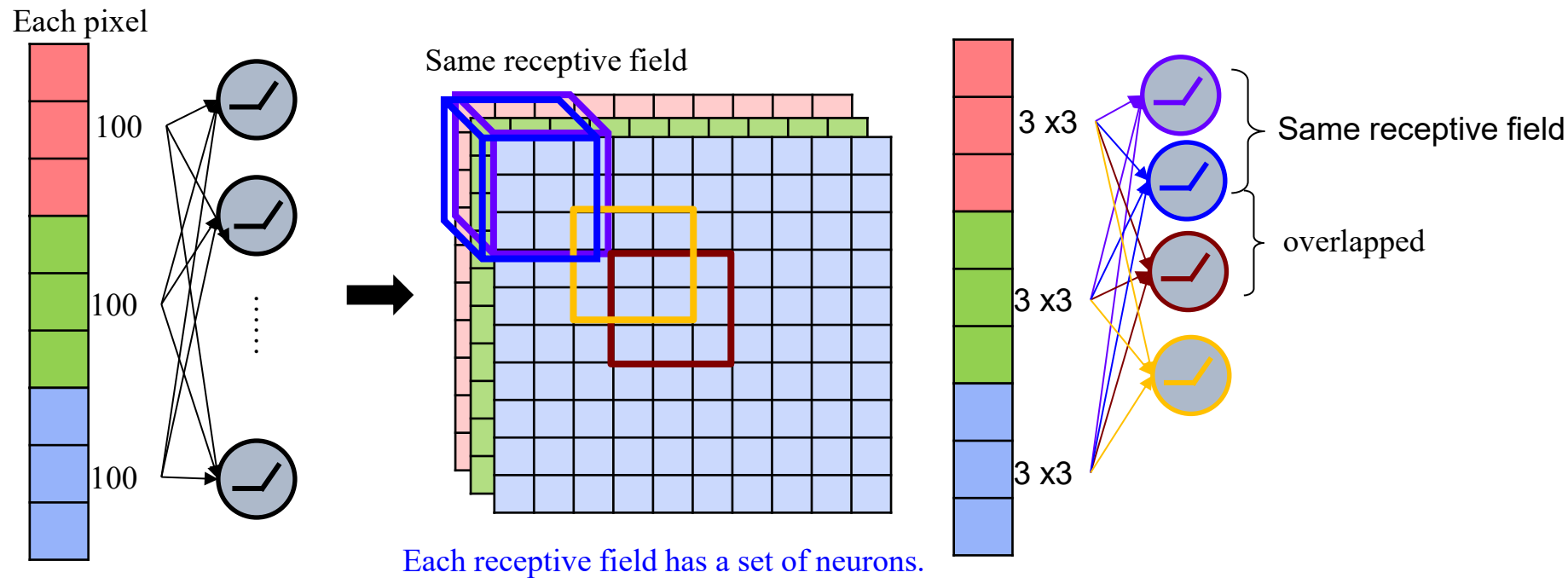


receptive field of s_3



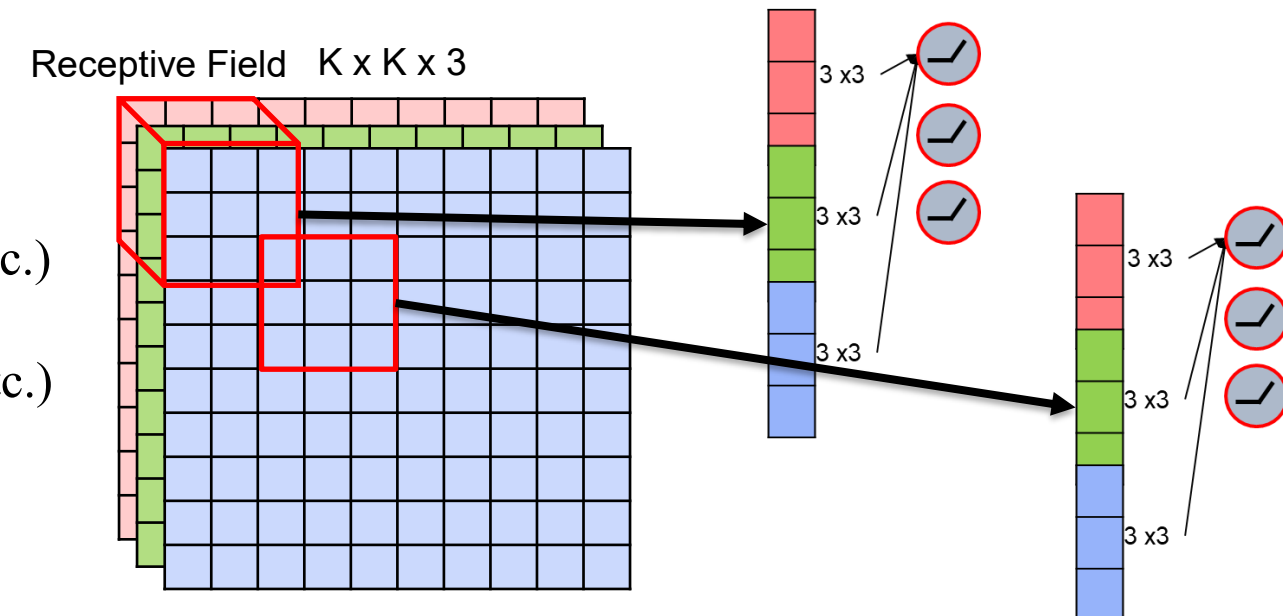
receptive field of g_3

Receptive Field



- **Convolution Filters (Kernels)**

1. Each filter scans across the spatial dimension
2. Each filter learns to detect **specific patterns** (edges, textures, etc.)
3. No strict rule — depends on task complexity and input size (covering one channel to restricted connectivity, non-square, etc.)
4. Typical settings:
 - a. One weight over all channel
 - b. Kernel size (e.g., 3 x 3)

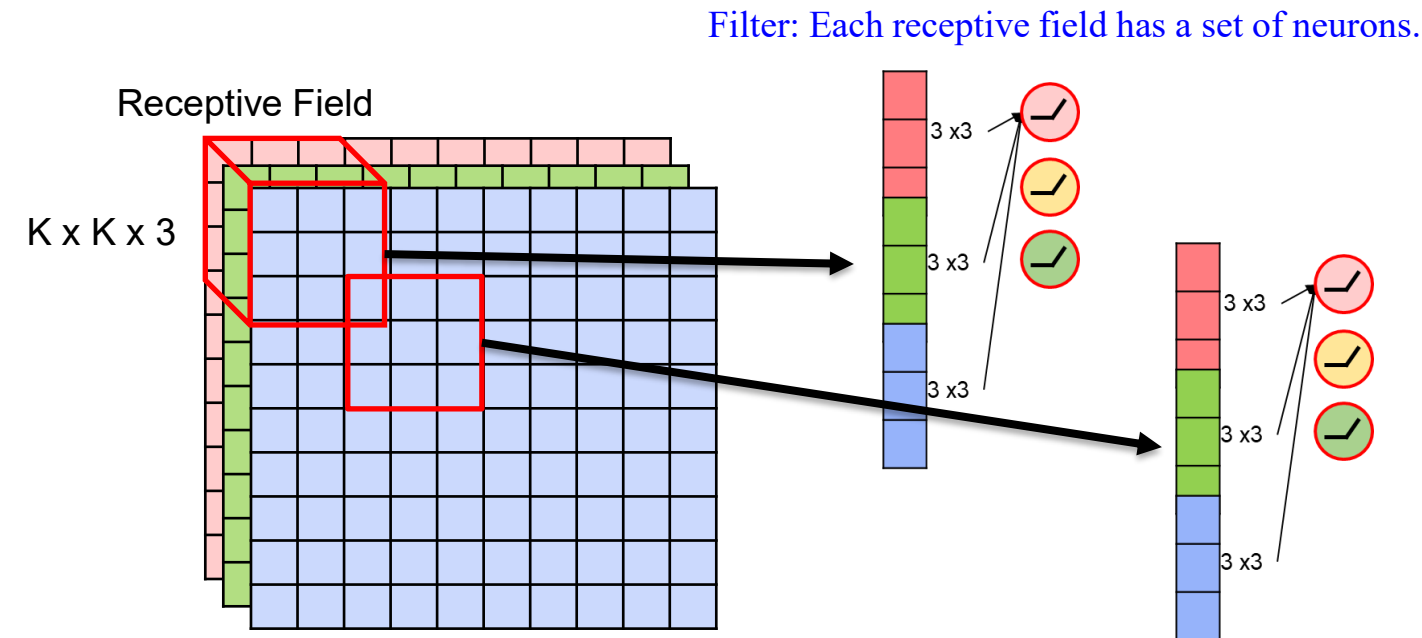




Receptive Field

- **Translation (Parameter Sharing):**

The same filter parameters are shared across locations, allowing detection of the same feature regardless of position.



- two neurons possessing *identical receptive-field locations* (in terms of spatial extent) would each have **distinct sets of parameters**, resulting in a **locally connected layer** rather than a true convolutional layer.

CNN's Key Ideas (Convolutional Neural Networks)



- Parameter Sharing
 1. The **same set of weights** (kernel) is applied across **all spatial locations**.
 2. Each parameter is **reused** many times → **fewer parameters, better generalization**.
 3. Greatly improves **statistical efficiency**.
- Equivariance
 1. Convolution is **equivariant to translation**:
if the input shifts, the output shifts in the same way.
 2. Ensures the network preserves **spatial relationships**.
 3. Foundation for capturing **structure** in images.



What is Convolution? (卷積/褶積)

- **Convolution** is an operation on **two functions** that produces a **third function** expressing how one function modifies or overlaps with the other.
- Suppose we track the position of a spaceship over time with a **noisy sensor** $x(t)$.
To smooth the readings, we apply a **weighted average** of nearby values (locality), using a weighting function $w(a)$ (e.g., recent values are more important).

$$s(t) = \int x(a) w(t - a) da = (x * w)(t) \quad \text{weighted sum of shifted inputs}$$

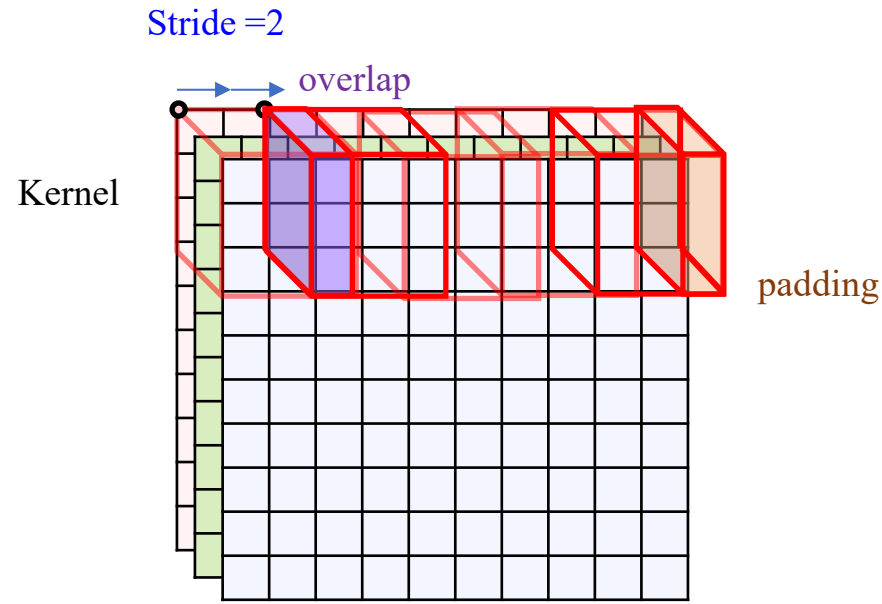
Discrete Convolution

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a) w(t - a)$$

- In machine learning, we often apply convolution between:
 - An input signal or image x
 - A kernel or filter w
 - The result is an output feature (特徵) map (e.g., texture, pattern)



Receptive Field



Convolution Operation

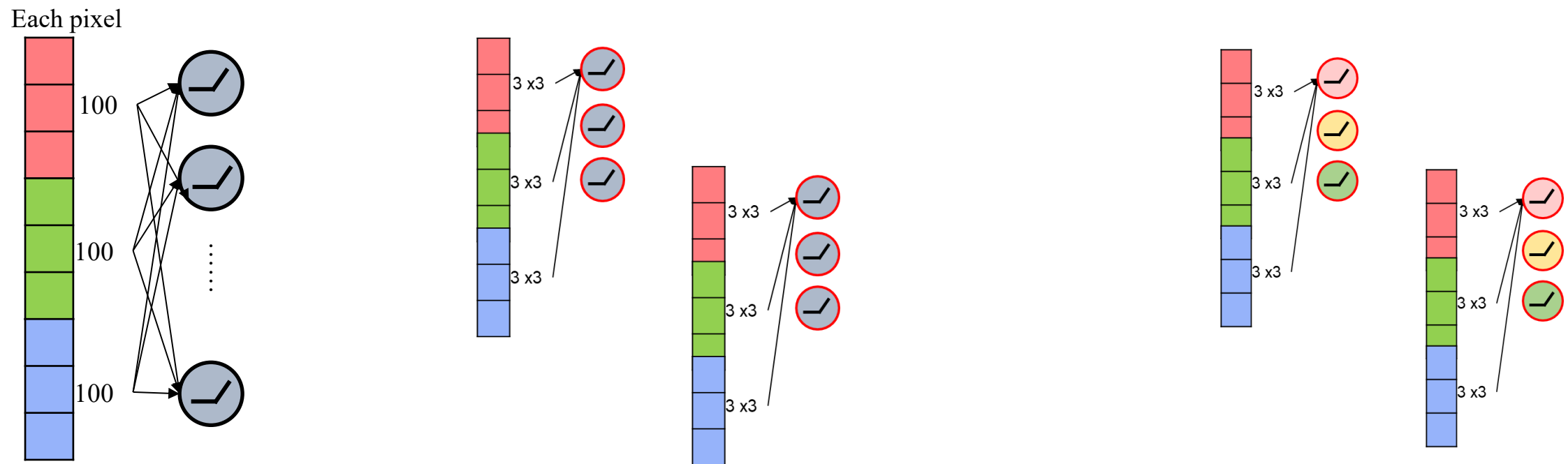
- Kernel size: Size of the convolving kernel
- Stride: Step size of the filter when sliding
- Padding: Add zeros around the input
- Spacing between kernel elements. Default: 1
- **Spatial size** of output (Pytorch Conv2d):

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor$$

Model Capacity and Parameter Efficiency

- Capacity (Number of trainable parameters):
 - Higher capacity may be higher risk of overfitting
 - Fully Connected Layer > Receptive Field (Local Connectivity) > Parameter Sharing (CNN)

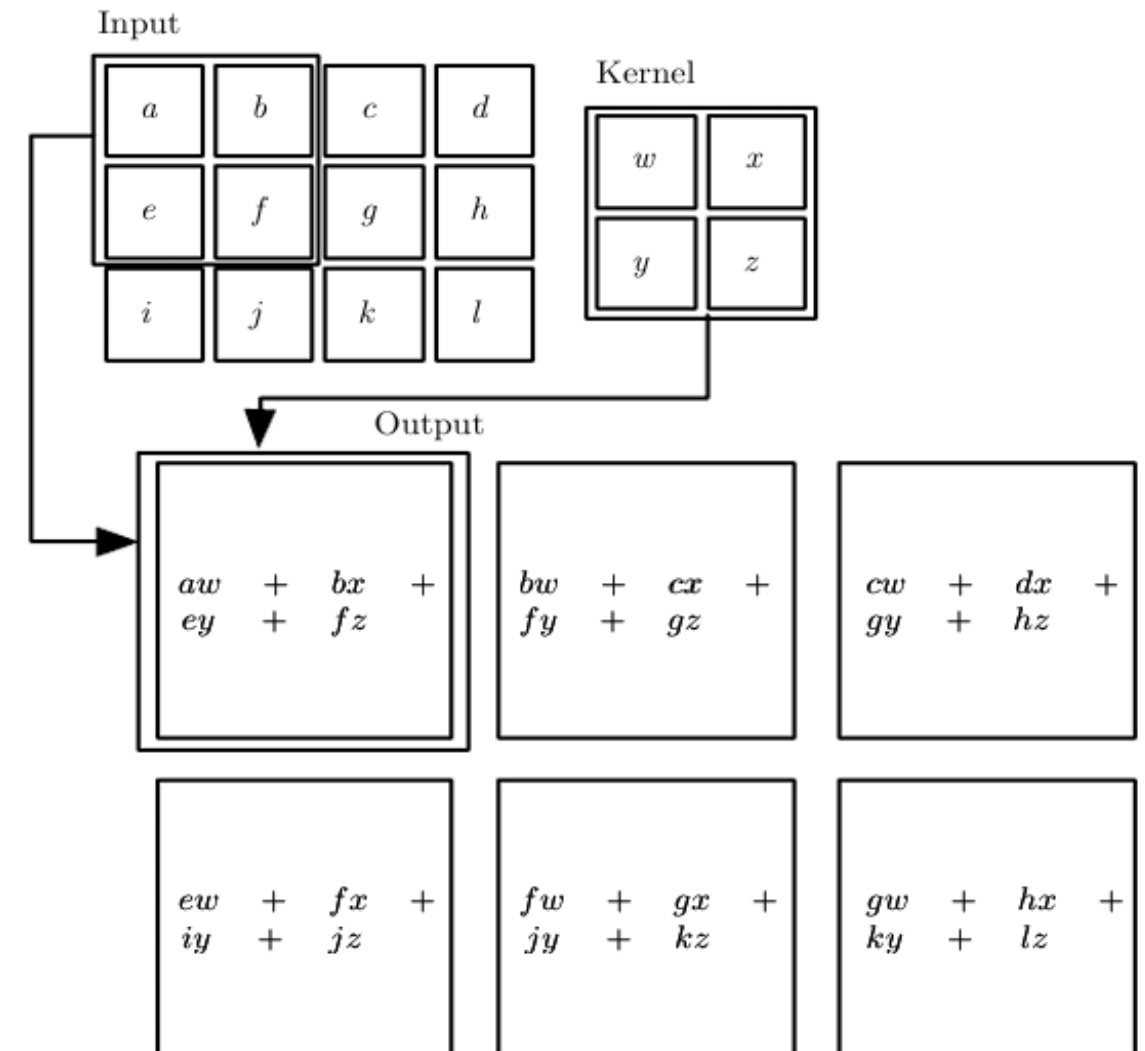




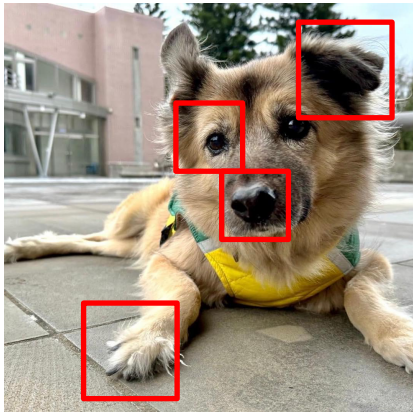
Convolution Layer

- The convolution layer computes a weighted sum over a local receptive field.
- For each output position:
 - Extract** a local patch from the input.
 - Compute the dot product** between the patch and the **kernel (filter)**.
 - Sum** over all channels (depth dimension).
 - Add bias** and apply activation function.

$$y(i, j) = \sigma \left(\sum_{c=1}^{C_{in}} \sum_{m,n} W_{m,n,c} \cdot X_{i+m,j+n,c} + b \right)$$



Convolution Layer



6 x 6 image

Stride =1
Shift filters →

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Weightings in the filter are the **parameters to be learned**

Feature map: Specify **the location of the feature** in the original image.

Filter 1 (for pattern 1)

1	-1	-1
-1	1	-1
-1	-1	1

Convolution

Feature map 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

4 x 4 image
with M channels

Filter 2 (for pattern 2)

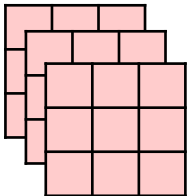
-1	1	-1
-1	1	-1
-1	1	-1

Convolution

Feature map 2

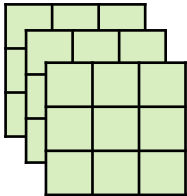
-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Filter 1



K x K x channel
tensor (> 2D matrix)

Filter 2



K x K x channel
tensor (> 2D matrix)

⋮

⋮

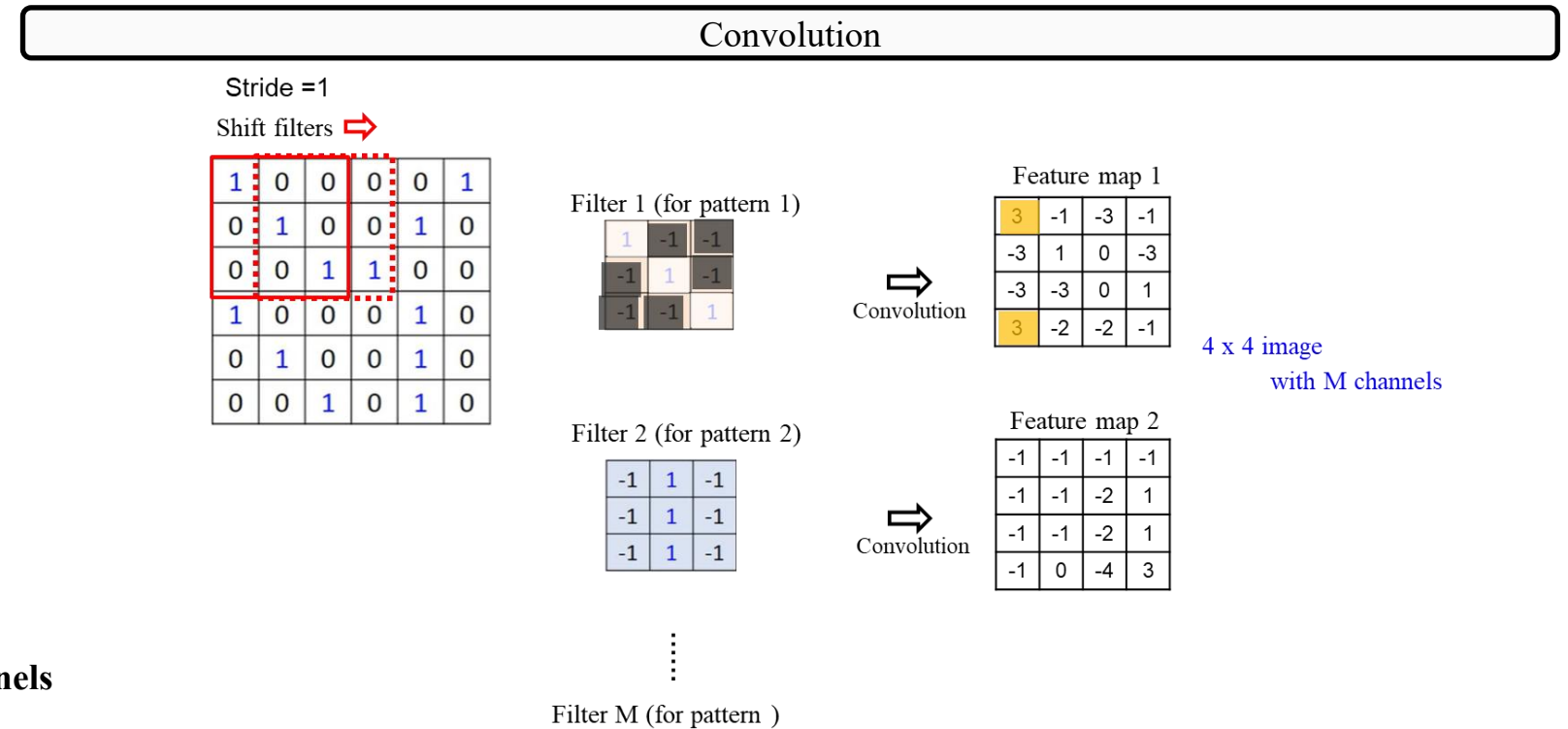
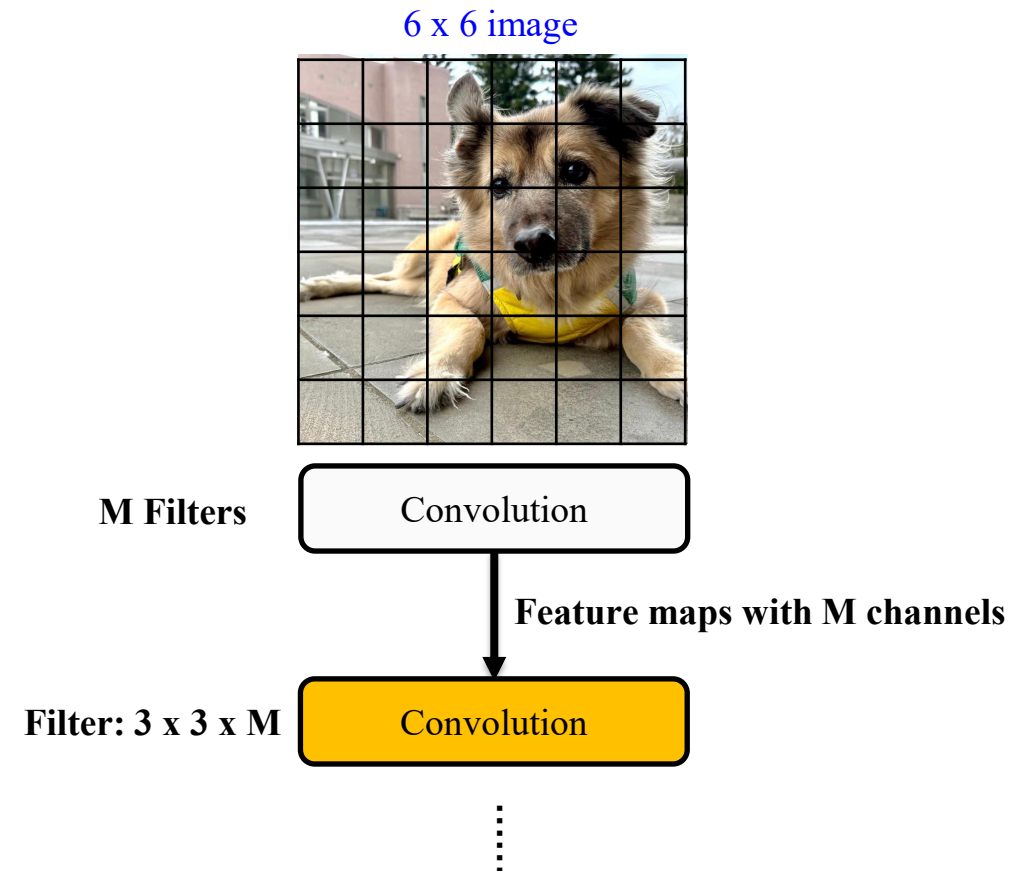
Filter M (for pattern)

-1	-1	-1	-1	1	0
1	1	1	-1	1	0
0	0	0	-1	1	0



Convolution Layer

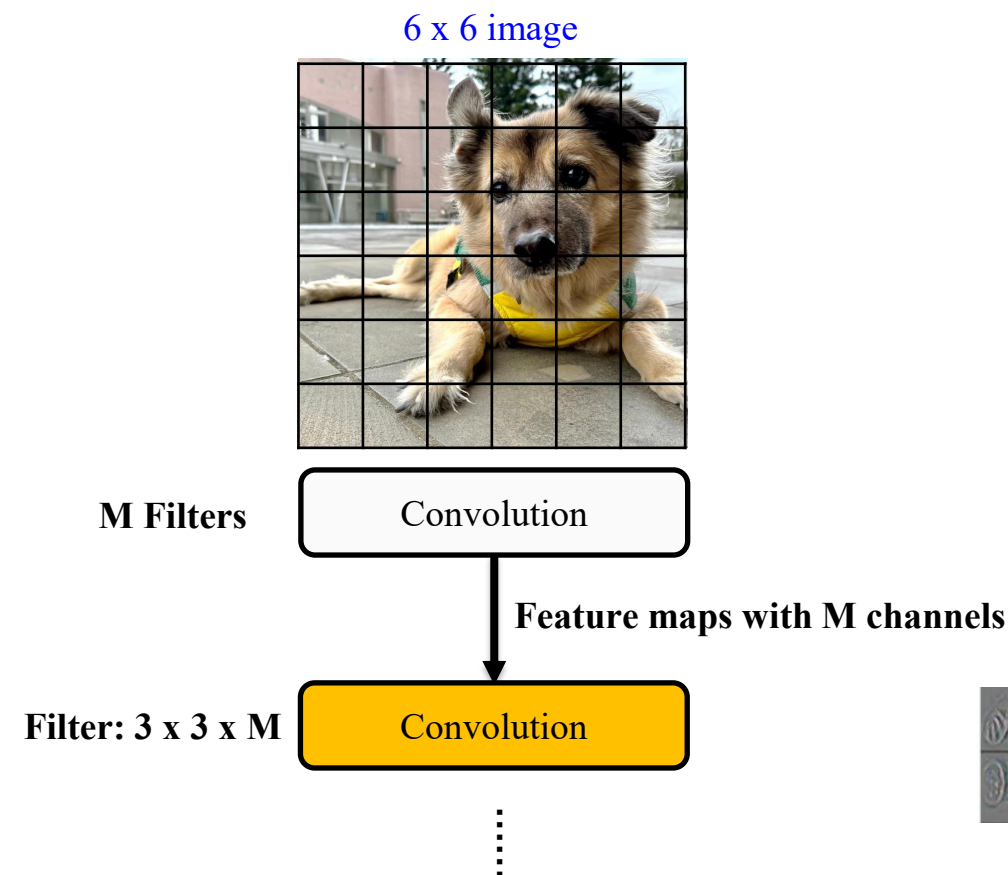
- Process



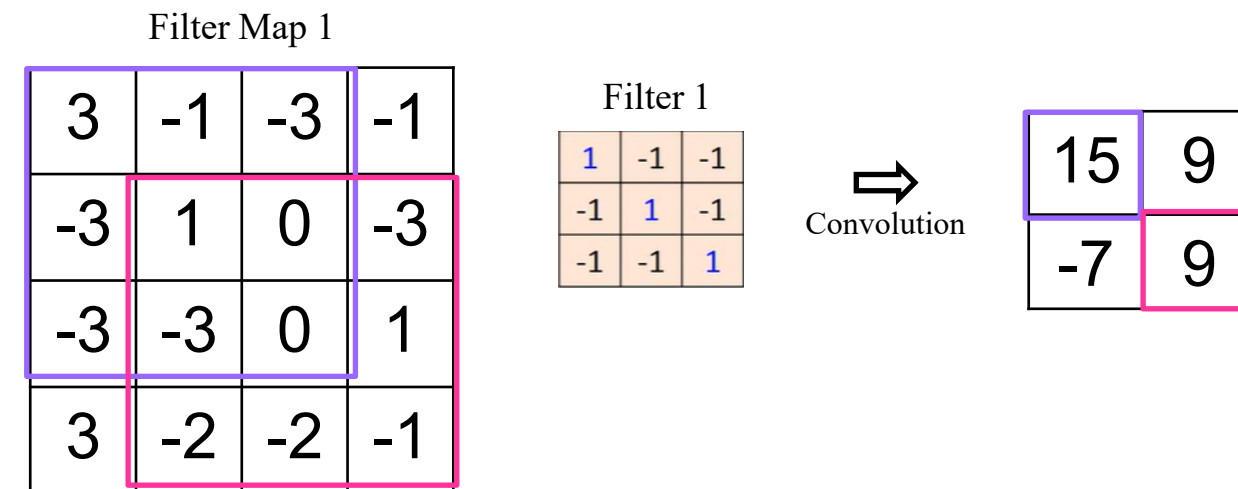


Convolution Layer

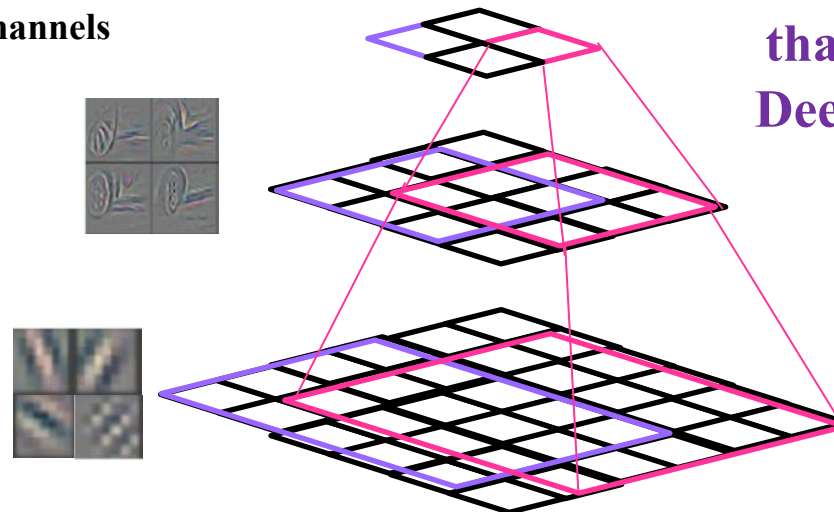
- Process



Convolution



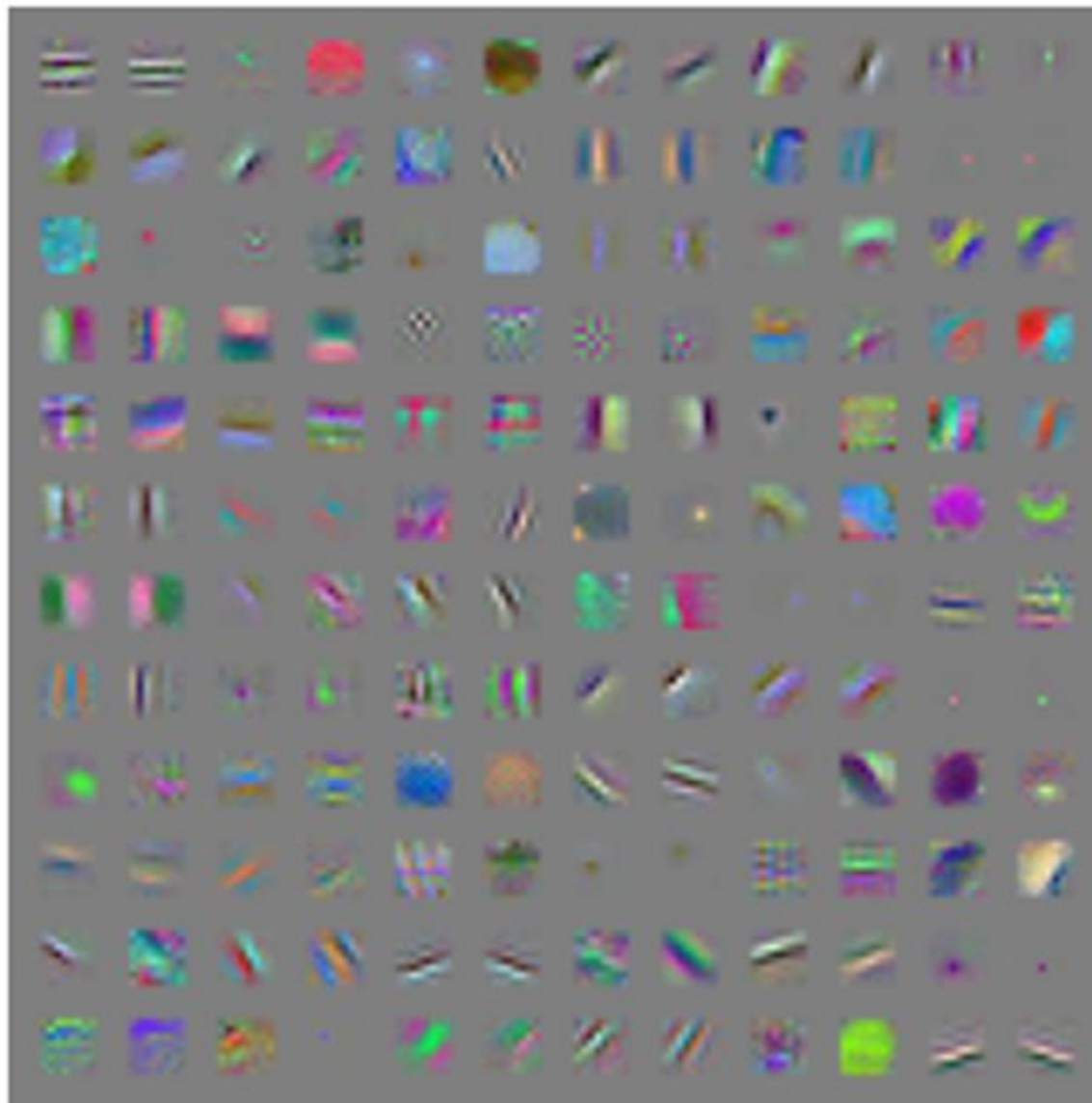
Deep filters have larger receptive field than shallow filter.
Deep filters allow more complex patterns



Convolution Layer



- Convolution kernels





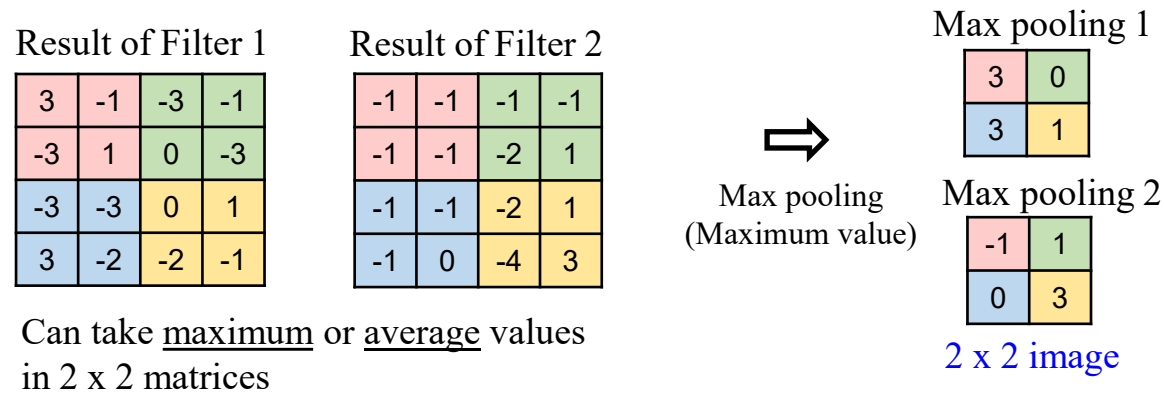
Hidden Layer

- Convolution Stage (Linear Operation)
Applies a set of learnable linear filters (kernels) to local receptive fields of the input.
Each output unit computes an affine transform (weighted sum + bias)
- Detector Stage
Introduces **nonlinearity** into the model, enabling it to represent complex functions.
- Subsampling / Aggregation (Pooling)
Aggregates **neighboring activations** to produce **summary statistics**
Reduces **spatial resolution** and **model complexity**



Max Pooling

- Subsampling the pixels will not change the object.
- A max pooling layer does not introduce new weights to learn.



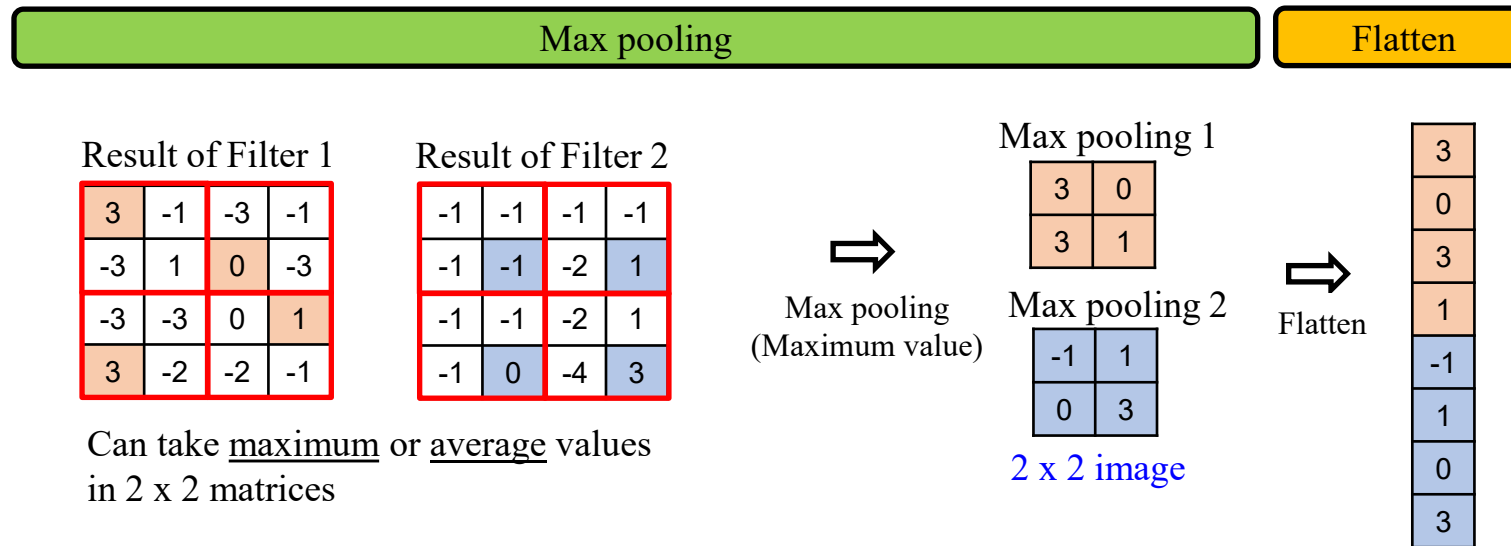
Max pooling makes feature map invariant to input translation

- This enforces the belief that **exact spatial position is less important** than the presence of a feature in a local neighborhood.
 - The network becomes invariant to small translations of the input.
- Imposing an **infinitely strong prior** that the learned function should be local (for convolution) and invariant to small translations



Max Pooling

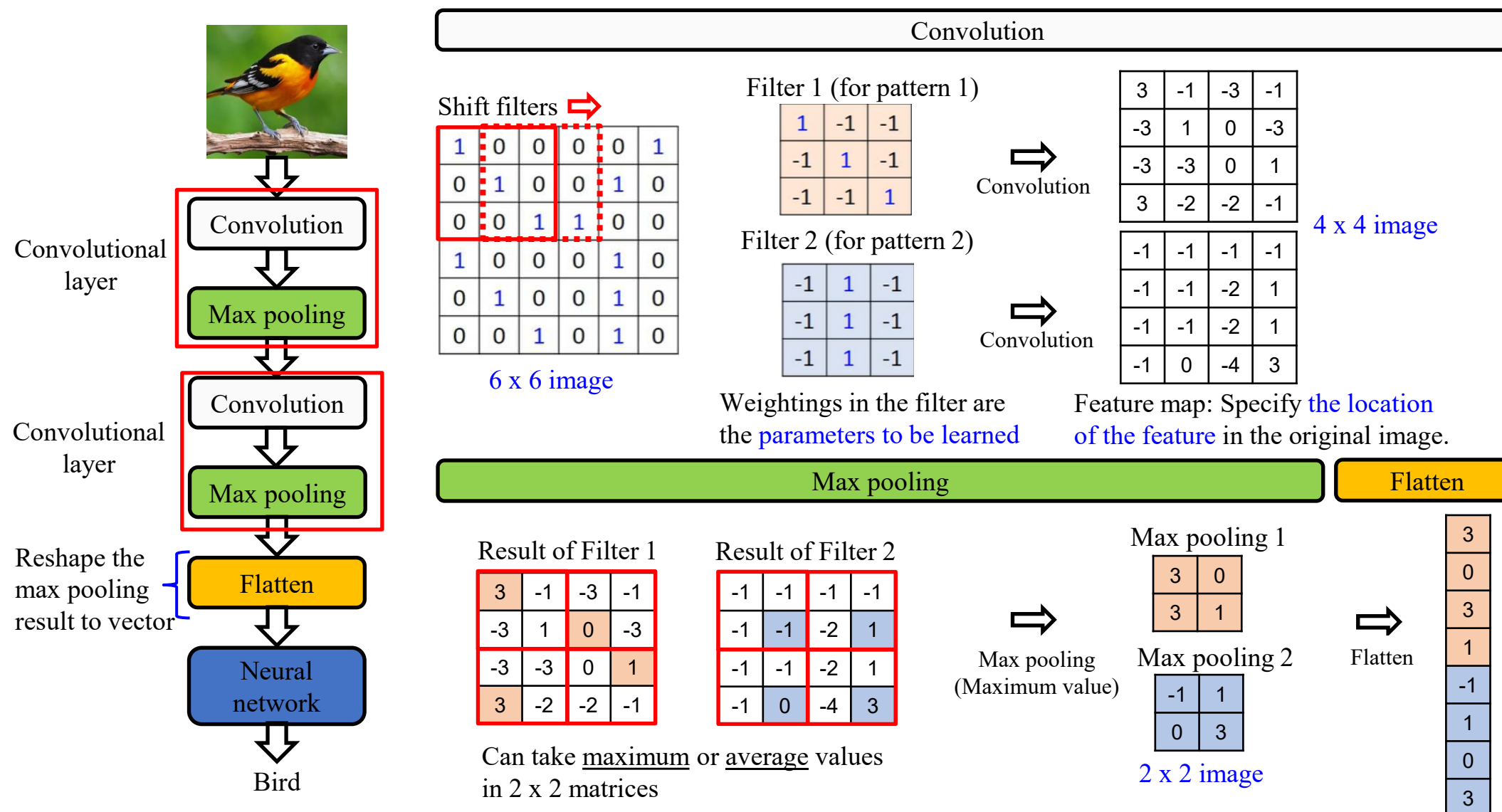
- Subsampling the pixels will not change the object.



Convolutional neural network



- CNN process



Spatial Transformation?

- Translation?
 - Rotation?
 - Scaling?
- Data augmentation

